

ДОКУМЕНТАЦИЯ НА ПРОЕКТ

GIANT

SUPER-GIANT - система за подготовка, трениране и ускорен инференс на езикови
модели

Автор Антон Иванов Христов

Ръководител Евгени Драгомиров Димов

София, 2026

1. ТЕМА

Разработване на цялостна инженерна система за обучение, оценка и ускорен inference на decoder-only езикови модели, включително изследователски слой за хибридно decoding поведение (Anchor-TiDAR), оптимизация на KV cache политика и анализ на throughput/latency при реални GPU ограничения.

2. АВТОРИ

Антон Иванов Христов

- ЕГН: 0751296460
- Адрес: гр. София
- Телефон: 0887776969
- Имейл: anton.i.hristov.2021@elsys-bg.org
- Училище: Технологично училище “Електронни системи” към ТУ - София
- Клас: 12 клас

3. РЪКОВОДИТЕЛ

Евгени Драгомиров Димов

- Телефон: 0886694906
- Имейл: evgeni.dimov@ocado.com
- Длъжност: Софтуерен инженер, Ocado Technologies
- Роля в проекта: научен ръководител

4. РЕЗЮМЕ

Проектът GIANT + Anchor-TiDAR представлява практическа и изследователска платформа за modern LLM development lifecycle: от data ingestion и training до inference benchmarking и експерименти с хибридни decode механизми. Реализацията е ориентирана към reproducible engineering execution чрез конфигурации, checkpointing, стандартизирани entry points и детайлно логване на резултатите.

Платформата е разработена така, че да обслужва две нужди едновременно: (1) стабилно обучение и използване на работещ модел в production-like CLI среда и (2) бързо тестване на нови изследователски идеи при контролирани условия. В резултат системата е приложима както за научно-изследователска работа, така и за практическа експлоатация на модели.

4.1. Цели

Основната цел е изграждане на работещ и разширяем LLM stack, който да даде измерими резултати в три направления: качество на модела, скорост на inference и възпроизводимост на експериментите.

Подцелите са:

- да се реализира data pipeline за подготовка на големи текстови корпуси (ingest, cleaning, sharding, indexing);
- да се реализира надежден training loop с resume/curriculum механизъм и checkpoint recovery;
- да се изгради inference слой с KV cache optimization и измерване на tokens/s, latency, acceptance behavior;
- да се валидира sequence-level hybrid decoding подход (Anchor-TiDAR) с фокус върху Free Token Slots и end-to-end speedup;
- да се изведе практическа методология за сравнение на decode режими при различен хардуер и различни model scales.

Кратък анализ на потребностите показва, че в наличните open-source решения често липсва интеграция между training и high-quality benchmarking pipeline в единна среда. Обикновено има добри отделни компоненти, но не и обща, reproducible, end-to-end структура с лесна поддръжка на експериментален слой. Настоящият проект адресира именно тази празнина.

4.2. Основни етапи в реализирането на проекта

Проектът е реализиран от един автор, поради което всички дейности са изпълнявани последователно в единна инженерна рамка:

1. Дефиниране на архитектурни изисквания и структури на конфигурации.
2. Изграждане на data pipeline и тренировъчни datasets.
3. Реализация на training pipeline за decoder-only Transformer.

4. Реализация на inference pipeline с KV cache optimization.
5. Добавяне на TiDAR/Anchor-TiDAR изследователски модул.
6. Провеждане на benchmark кампания и анализ на резултатите.
7. Документиране, верификация и подготовка на финални артефакти.

Ролята на научния ръководител е консултативна: насочване към по-добри инженерни практики, критерии за валидиране на резултатите и структуриране на експерименталната методология.

4.3. Ниво на сложност на проекта

Нивото на сложност е високо, защото системата комбинира няколко трудни слоя едновременно:

- **Numerical complexity** - работа с големи тензори, mixed precision режими, JAX/XLA compilation constraints и memory-bound behavior.
- **System complexity** - синхронизация между local CPU workflow и remote GPU execution, reproducible environments чрез Docker, надеждна работа с артефакти.
- **Algorithmic complexity** - едновременно поддържане на класически autoregressive decode и sequence-level hybrid decode с допълнителна masking логика.
- **Evaluation complexity** - коректно измерване на speedup, latency, acceptance и quality indicators без да се смесват compile overhead, warmup ефекти и run-to-run variance.

Проектът решава практически значим проблем: как един и същ модел да се използва едновременно за високо качество и за висок inference throughput, без задължителна зависимост от втори “draft” модел и без нарушаване на устойчивия инженерен цикъл.

4.4. Логическо и функционално описание на решението

Системата е модулна и е организирана в два главни слоя:

- **GIANT Core** - стабилен training/inference stack.
- **TiDAR Extension** - изследователски слой за hybrid decode и допълнителни loss/mask механизми.

Взаимодействието между модулите е организирано чрез shared конфигурации, стандартизирани entry points и общ artifact storage модел.

4.4.1. Базова архитектура на модела

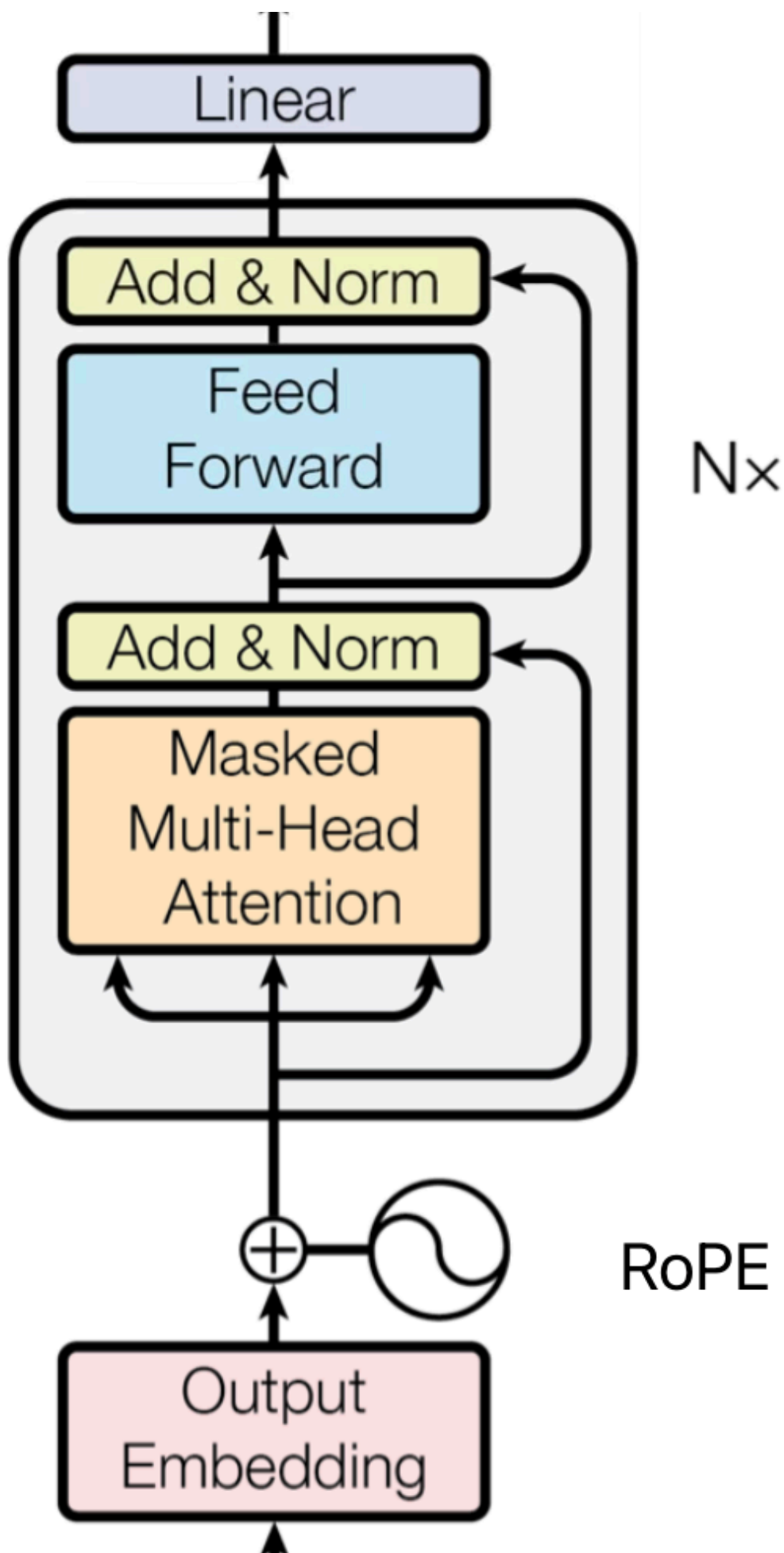


Figure 1: Decoder-only Transformer архитектура, използвана като базов модел

Базовият модел е decoder-only Transformer с causal masking. Всеки training sample се обработва като последователност от токени, а целевата функция е next-token prediction. Архитектурно са използвани съвременни компоненти като RMSNorm, RoPE и SwiGLU, което осигурява стабилност при обучение и добра ефективност при inference.

Self-attention операторът е:

$$A(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

където M е причинно-следствена маска, която гарантира, че позицията i вижда само токени с индекс $j \leq i$.

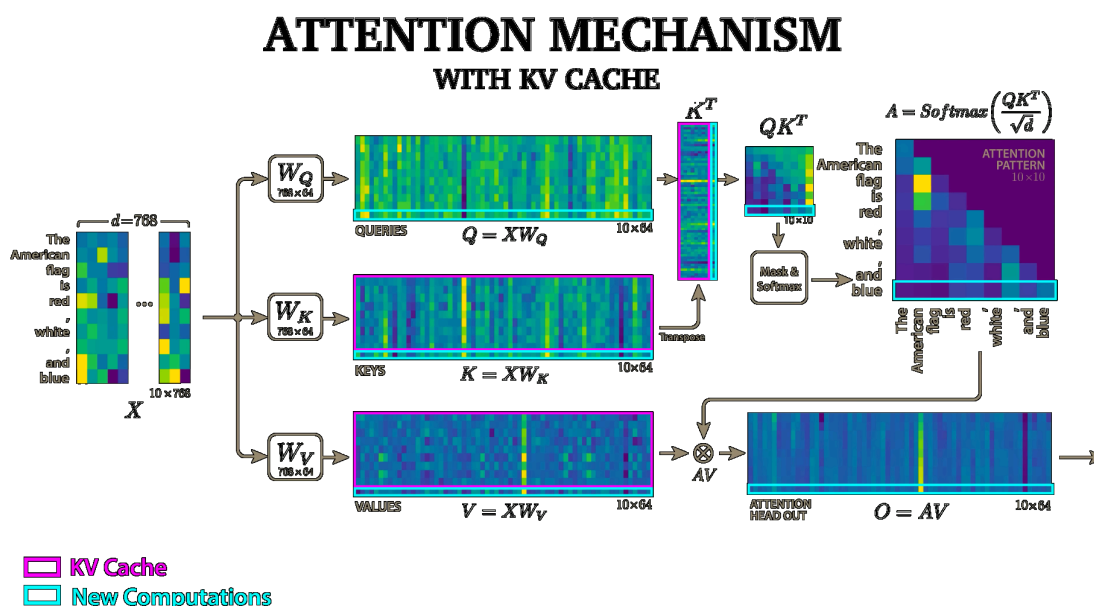


Figure 2: Attention механизъм и контекстно претегляне

4.4.2. Функционални модули и зависимости

Функционално решението е изградено от следните модули:

- **Data Module** - ingest, cleaning, dataset sharding, статистики, проверка на входните файлове.
- **Training Module** - optimizer update loop, curriculum stages, gradient accumulation, checkpoint save/recover.
- **Inference Module** - prefill + decode път, sampling режими, KV cache management, performance counters.
- **Evaluation Module** - benchmark сценарии, автоматично събиране на метрики, сравнение между конфигурации.
- **Deployment Module** - Docker runtime, remote execution, data sync, artifact management.

Комуникацията между модулите е конфигурационно-ориентирана: модулите не разчитат на ръчно редактирани вътрешни параметри, а на explicit YAML settings, което намалява риска от скрити зависимости.

4.4.3. Data и training pipeline

Data pipeline обработва корпусите в последователни стъпки: нормализация, филтриране, токенизация и shard-ване. Получените shard файлове се използват директно от training loop-a, така че I/O достъпът да е предвидим и подходящ за дълги run-ове.

Training pipeline е stage-based. Всеки stage има собствени параметри (контекстна дължина, learning schedule, loss weights), като състоянието се съхранява чрез checkpoint. Така при прекъсване обучението може да продължи без загуба на прогрес.

Загубата за autoregressive обучение е:

$$L_{AR} = -\frac{1}{N} \sum_{t=1}^N \ln p(y_t | y_{<t})$$

Тази формулировка осигурява стабилен базов критерий за оценка преди включване на допълнителни изследователски термини.

4.4.4. Anchor-TiDAR: hybrid decode логика

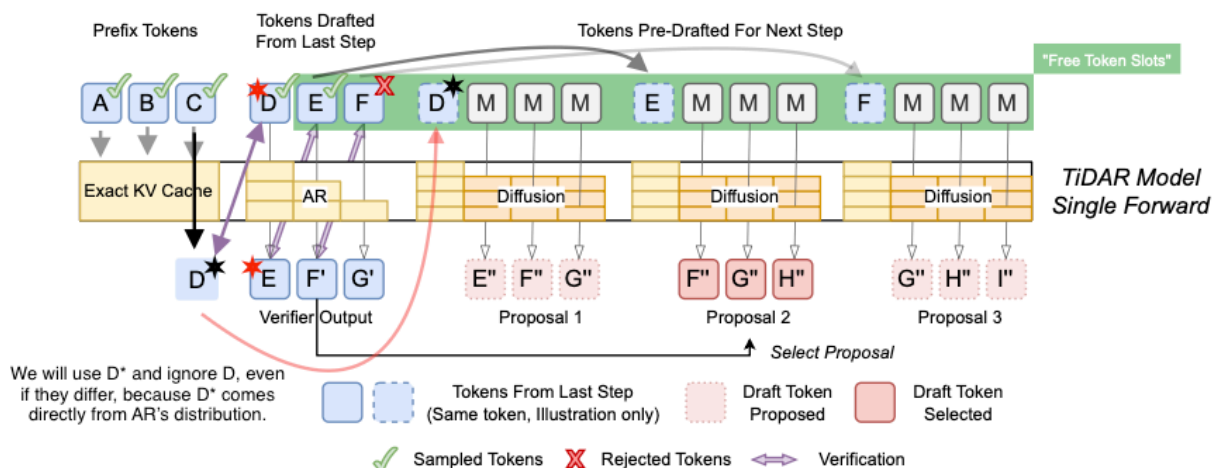


Figure 3: Anchor-TiDAR forward pass: verify + predraft в един моделен проход

Anchor-TiDAR въвежда sequence-level hybrid decode: системата генерира draft предложения паралелно и ги верифицира autoregressively в рамките на един forward pass със специализирани маски. Това позволява по-добро използване на GPU паралелизма спрямо класическия AR decode път.

Логически decode цикълът е:

1. prefill на контекста и инициализация на KV cache;

2. генериране на draft кандидати;
3. verify стъпка и приемане/отхвърляне на префикс;
4. commit/rollback на pointer-и в KV cache;
5. преход към следваща итерация.

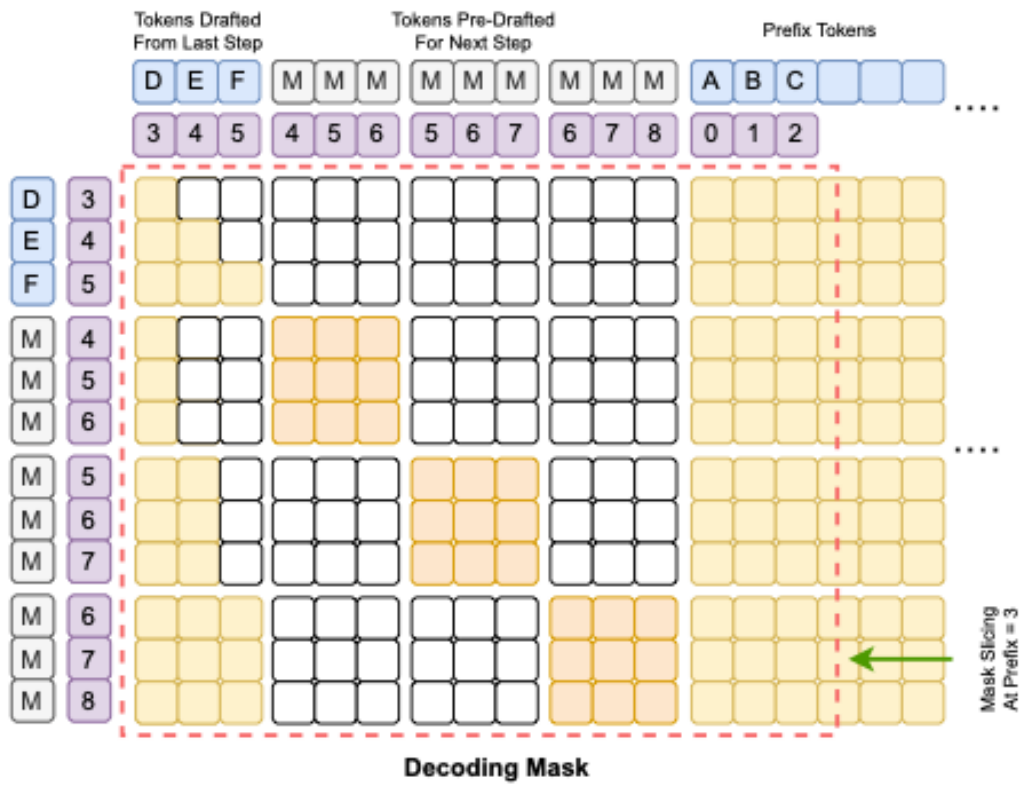


Figure 4: Структурирана attention маска за TiDAR inference

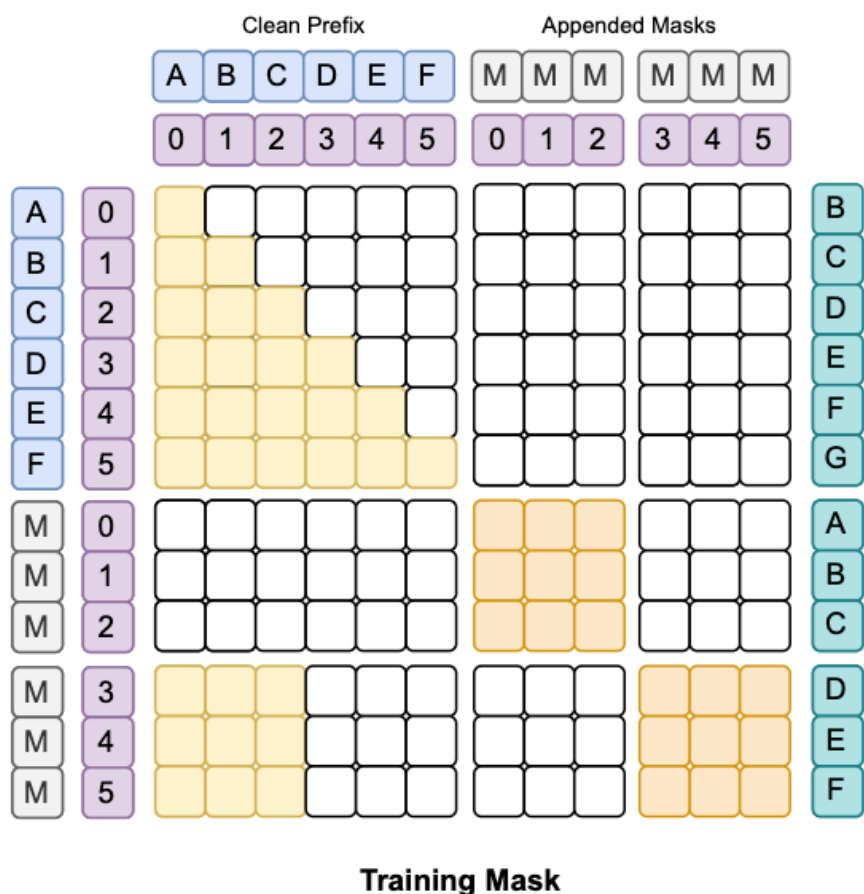


Figure 5: Training маска за TiDAR режим

Практическата полза е, че verify/predraft изчисленията се амортизират в един компактен execution path, което намалява относителния overhead при decode.

4.4.5. Free Token Slots и GPU ефективност

Концепцията Free Token Slots описва случаи, при които в дадена decode итерация има неизползван паралелен ресурс, който може да бъде запълнен с полезна допълнителна работа. Този анализ е ключов за latency-critical deployment сценарии, защото показва къде архитектурните промени носят реална speedup стойност.

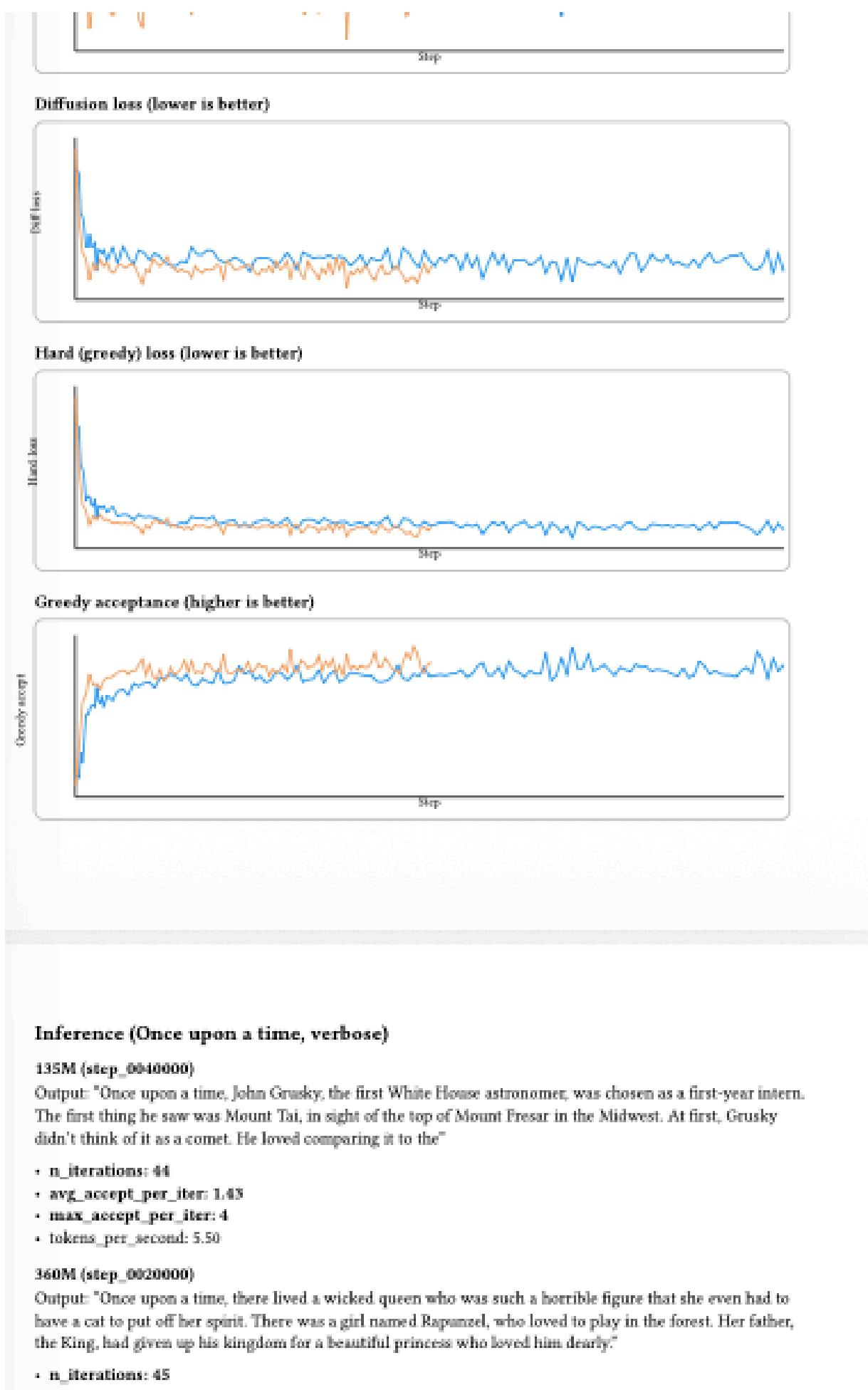


Figure 6: Примерни експериментални графики за decode поведение и speedup анализ

Валидираните резултати в този проект показват, че hybrid decode подходът е устойчив при мащаби до 7B параметри и може да осигури значимо ускорение спрямо AR baseline при запазване на конкурентно качество.

4.5. Реализация

Изборът на технологични средства е направен с фокус върху надеждност и ефективност:

- **Python** като основен език за бърза разработка и богата ML екосистема.
- **JAX/Flax/Optax/Orbax** за висока числена производителност, модулност и устойчив checkpointing.
- **Docker** за reproducible runtime.
- **Remote GPU execution** за тежки training/inference run-ове.
- **S3-compatible workflow** за пренос и съхранение на големи артефакти.

Използваните алгоритми включват:

- decoder-only autoregressive training;
- speculative/hybrid decode стратегии;
- KV cache bucket policy;
- анализ на acceptance и локални logits профили;
- loss composition за TiDAR post-training sweep.



Figure 7: Containerized execution среда за training и inference

Използваната литература включва фундаменталните Transformer публикации, работи за serving оптимизации и референтната статия за TiDAR.

4.6. Описание на приложението

4.6.1. Стартиране и инсталация

Приложението се стартира в Linux/macOS среда с Python и Docker. Типичният процес е:

1. клониране на репозиторията;
2. настройка на dependencies/venv или Docker контейнер;
3. проверка на конфигурационните файлове;
4. стартиране на data pipeline, training или inference entry point.

Примерни команди:

```
python GIANT/v2/data_pipeline/build_corpus.py
```

```
python GIANT/v2/model/Run_training.py --resume latest
```

```
python GIANT/v2/model/Generate_faster.py --prompt "Hello" --steps 128
```

```
python TiDAR/model/Run_training.py --config TiDAR/model/Config_135m.yml
```

```
python TiDAR/model/inference.py --prompt "Demo" --draft_len 8
```

4.6.2. Използване

Потребителят работи изцяло през CLI, като управлява run параметрите чрез YAML конфигурации и command flags. Основните режими са:

- подготовка на данни;
- обучение (базов режим или TiDAR режим);
- inference (AR или Anchor-TiDAR);
- benchmarking и анализ на метрики.

4.6.3. Поддръжка

Поддръжката включва:

- версия на конфигурации;
- периодично архивиране на checkpoints и логове;
- проверка на съвместимост между model state и tokenizer;
- повторно възпроизвеждане на benchmark run-ове при промени в кода.

Тази организация позволява системата да се използва продължително във времето, без загуба на проследимост между версия на модела, входни данни и получени резултати.

4.7. Заключение

Проектът постига основната си цел: реализирана е цялостна, работеща и разширяема платформа за обучение и inference на големи езикови модели, която може да служи както за практически deployment сценарии, така и за изследователски експерименти.

Най-същественият резултат е, че в рамките на една единна codebase са съчетани стабилен training stack и валидиран hybrid decode подход с измеримо ускорение спрямо класически AR baseline. Това дава реална стойност за latency-sensitive приложения и обоснована основа за следващи изследвания.

Към момента платформата има директно приложение като вътрешна инженерна среда за разработка, сравнение и валидиране на LLM конфигурации. Възможностите за развитие включват multi-GPU training, serving layer за по-висок паралелизъм на заявки, по-мощни evaluation кампании и разширяване на model scales към по-големи production профили.

Приложение А. Примерен потребителски интерфейс (CLI)

Следващият пример илюстрира реалната употреба на приложението от гледна точка на краен технически потребител. Интерфейсът е команден (CLI), като всички критични действия се изпълняват чрез ясни и повторяеми команди.

```
# 1) Подготовка на корпус
python GIANT/v2/data_pipeline/build_corpus.py

# 2) Стартиране/продължаване на обучение
python GIANT/v2/model/Run_training.py --resume latest

# 3) Бърз inference тест
python GIANT/v2/model/Generate_faster.py --prompt "Once upon a time" --steps
128

# 4) TiDAR експериментален режим
python TiDAR/model/Run_training.py --config TiDAR/model/Config_135m.yml
python TiDAR/model/inference.py --prompt "In the forest" --draft_len 8
```

Listing 1: Примерен CLI сценарий за стартиране на основните модули

Този подход осигурява висока прозрачност и контрол: всяка команда е проследима, параметрите са експлицитни, а резултатите (логове, checkpoints, метрики) могат да бъдат директно архивирани и възпроизведени в следващ run.